

Università degli Studi di Bari Aldo Moro

Corso di Laurea in Informatica

Metodi Avanzati di Programmazione

A.A. 2025/2026

Regression Tree Miner

Manuale utente e guida all'installazione

Progetto esteso

Studente: Giovane Francesco Alessandro

Matricola: 799104

Email: f.giovane@studenti.uniba.it

Indice

1	Introduzione	1
1.1	Scopo del progetto	1
1.2	Requisiti di sistema	2
1.3	Contenuto della distribuzione	3
2	Architettura del sistema	4
2.1	Panoramica	4
2.2	Package <code>bot</code>	5
2.3	Package <code>server</code>	6
2.4	Package <code>data</code>	6
2.5	Package <code>database</code>	7
2.6	Package <code>tree</code>	7
3	Installazione e avvio	8
3.1	Preparazione	8
3.2	Avvio di MySQL	8
3.2.1	Controllo del servizio su Linux	9
3.2.2	Controllo del servizio su Windows	9
3.2.3	Creazione automatica del database	9
3.2.4	Verifica della configurazione	10
3.3	Avvio del server	11
3.3.1	Avvio del server su Linux	11
3.3.2	Avvio del server su Windows	12
3.4	Avvio del bot Telegram	12
3.4.1	Avvio del bot su Linux	12
3.4.2	Avvio del bot su Windows	13

3.4.3	Connessione del bot a un server remoto	13
4	Utilizzo del bot Telegram	14
4.1	Avvio di una sessione	14
4.2	Comandi disponibili	15
4.3	Apprendimento di un nuovo albero	16
4.4	Predizione guidata	17
4.5	Caricamento di un albero salvato	18
4.6	Chiusura della sessione Telegram	19
4.7	Utilizzo del comando <code>/help</code>	20
4.8	Utilizzo del comando <code>/info</code>	20
5	Chiusura dell'applicazione	22
5.1	Arresto del bot	22
5.2	Arresto del server	22
5.3	Conservazione degli alberi	22

Elenco delle figure

2.1	Relazioni tra i package del progetto esteso.	5
2.2	Diagramma delle classi del package <code>bot</code>	5
2.3	Diagramma delle classi del package <code>server</code>	6
2.4	Diagramma delle classi del package <code>data</code>	6
2.5	Diagramma delle classi del package <code>database</code>	7
2.6	Diagramma delle classi del package <code>tree</code>	7
3.1	Avvio del server tramite script su Linux.	11
3.2	Avvio del bot Telegram tramite script su Linux.	13
4.1	Info bot prima dell'avvio.	14
4.2	Avvio di una sessione con <code>@RegressionTreeMinerBot</code> e visualizzazione del menu principale.	15
4.3	Apprendimento di un nuovo albero attraverso la chat Telegram.	17
4.4	Predizione guidata e visualizzazione della classe predetta.	18
4.5	Caricamento di un albero serializzato tramite Telegram.	19
4.6	Visualizzazione della guida tramite il comando <code>/help</code>	20
4.7	Visualizzazione delle informazioni sul progetto tramite il comando <code>/info</code>	21

Introduzione

1.1 Scopo del progetto

Regression Tree Miner è un'applicazione Java client-server finalizzata all'apprendimento e all'utilizzo di alberi di regressione a partire da dati memorizzati in un database MySQL.

Un albero di regressione è un modello predittivo che suddivide progressivamente gli esempi del training set in gruppi più omogenei, sulla base dei valori degli attributi esplicativi. Ogni nodo interno rappresenta una condizione applicata a un attributo discreto o continuo, mentre ogni nodo foglia contiene il valore numerico utilizzato come predizione.

Durante la fase di apprendimento, il sistema acquisisce gli esempi da una tabella del database e ricerca, per ciascun nodo, la suddivisione che riduce maggiormente l'errore quadratico degli esempi coperti. Il processo continua ricorsivamente fino alla costruzione completa dell'albero.

Il progetto esteso sostituisce il client a riga di comando con un bot Telegram. L'utente può quindi avviare una sessione, apprendere o caricare un albero ed effettuare predizioni direttamente da una chat Telegram.

Il progetto permette all'utente di:

- acquisire un training set da una tabella MySQL;
- costruire un albero di regressione;
- salvare l'albero appreso in un archivio serializzato con estensione `.dmp`;
- caricare un albero precedentemente salvato;
- effettuare predizioni guidate tramite Telegram;
- ripetere più predizioni utilizzando lo stesso albero;
- gestire sessioni indipendenti per più chat Telegram.

L'applicazione conserva l'architettura client-server del progetto base. Il bot Telegram svolge il ruolo di client: riceve i messaggi dell'utente, mantiene una sessione separata per ogni chat e inoltra le richieste al server tramite socket TCP. Il bot non accede direttamente al database e non costruisce direttamente l'albero.

Il server gestisce l'accesso a MySQL, l'apprendimento, il salvataggio, il caricamento degli alberi e l'esecuzione delle predizioni. Le risposte prodotte dal server vengono infine restituite all'utente nella chat Telegram.

1.2 Requisiti di sistema

Per eseguire il progetto sono necessari:

- un sistema operativo in grado di eseguire Java e MySQL;
- Java Runtime Environment o Java Development Kit 21 o superiore;
- MySQL 8 o compatibile;
- un terminale;
- una porta TCP libera per il server, impostata nel progetto al valore 8080;
- una connessione a Internet;
- un account Telegram;
- l'accesso al bot Telegram @RegressionTreeMinerBot;

Per verificare la presenza delle versioni corrette di Java e MySQL è possibile aprire un terminale ed eseguire:

```
java -version  
mysql --version
```

Se uno dei due comandi non viene riconosciuto, installare il componente mancante prima di procedere.

Su Linux Mint è possibile installare i pacchetti principali con:

```
sudo apt update  
sudo apt install openjdk-21-jdk mysql-server
```

1.3 Contenuto della distribuzione

La struttura logica del progetto esteso è la seguente:

```
progetto_esteso/  
|-- src/  
|   |-- bot/  
|   |-- server/  
|   '-- setup.sql  
|-- dist/  
|   |-- avvia_bot.bat  
|   |-- avvia_bot.sh  
|   |-- avvia_server.bat  
|   |-- avvia_server.sh  
|   |-- bot.jar  
|   |-- server.jar  
|   '-- telegram_token.txt  
|-- javadoc/  
|   |-- javadocForServer/  
|   '-- javadocForTelegramBot/  
|-- uml/  
|-- manuale_utente_progetto_esteso.pdf  
'-- casi_di_test_progetto_esteso.pdf
```

Architettura del sistema

2.1 Panoramica

Regression Tree Miner esteso utilizza la seguente architettura:

1. l'utente invia un messaggio al bot attraverso Telegram;
2. il bot riceve l'aggiornamento e identifica la chat dell'utente;
3. per ogni chat viene mantenuta una sessione indipendente;
4. la sessione stabilisce una connessione socket verso il server;
5. il server assegna la connessione a un thread dedicato;
6. per l'apprendimento, il server acquisisce i dati dal database MySQL tramite JDBC;
7. il package `tree` costruisce o carica l'albero di regressione;
8. durante la predizione, il server invia al bot una sequenza di domande;
9. il bot mostra le domande nella chat e inoltra al server le scelte dell'utente;
10. al raggiungimento di una foglia, il bot comunica la classe numerica predetta.

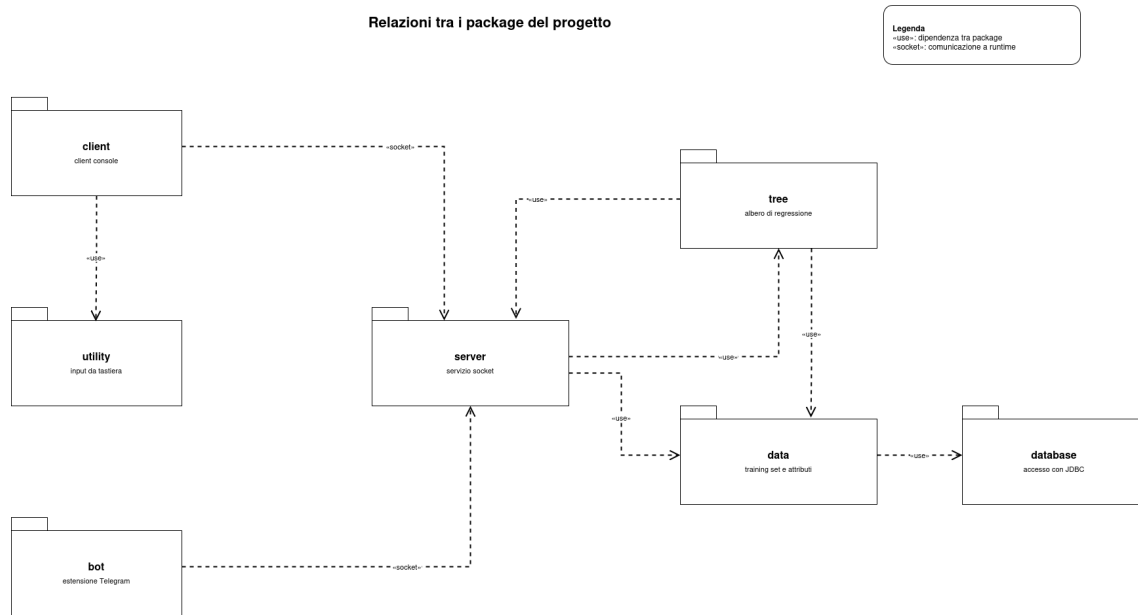


Figura 2.1: Relazioni tra i package del progetto esteso.

2.2 Package bot

Il package `bot` contiene il punto di avvio del bot Telegram, la gestione dei messaggi ricevuti e la comunicazione con il server. Per ogni chat Telegram viene creata una sessione distinta in modo da mantenere separati lo stato della conversazione e il percorso di predizione dei diversi utenti.

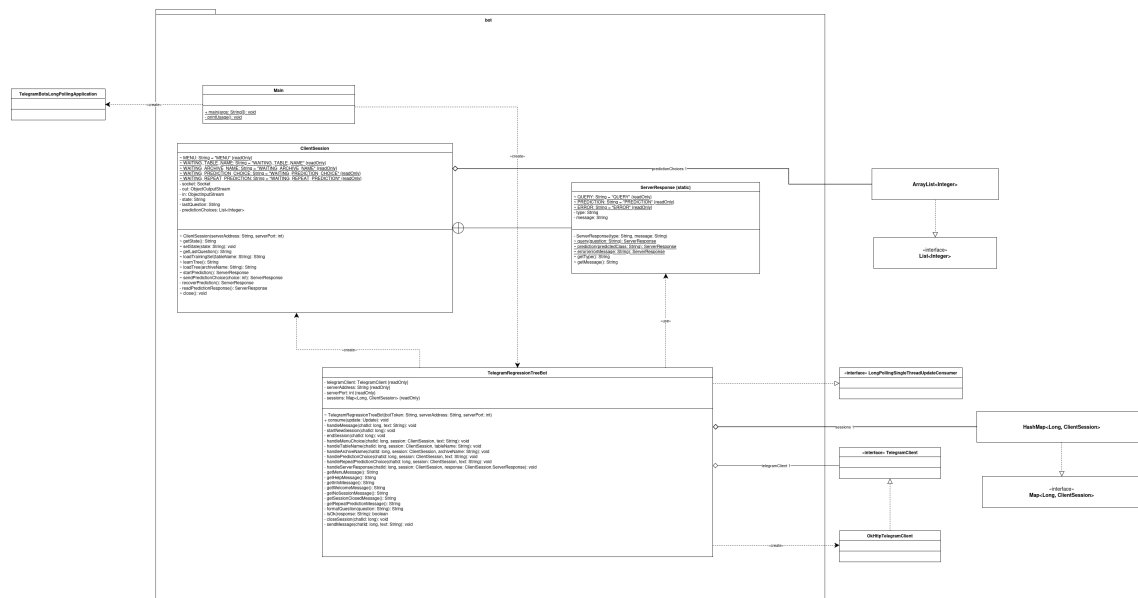


Figura 2.2: Diagramma delle classi del package `bot`.

2.3 Package server

Il package **server** gestisce la porta di ascolto, crea un thread per ogni connessione e coordina acquisizione del training set, apprendimento, caricamento e predizione.

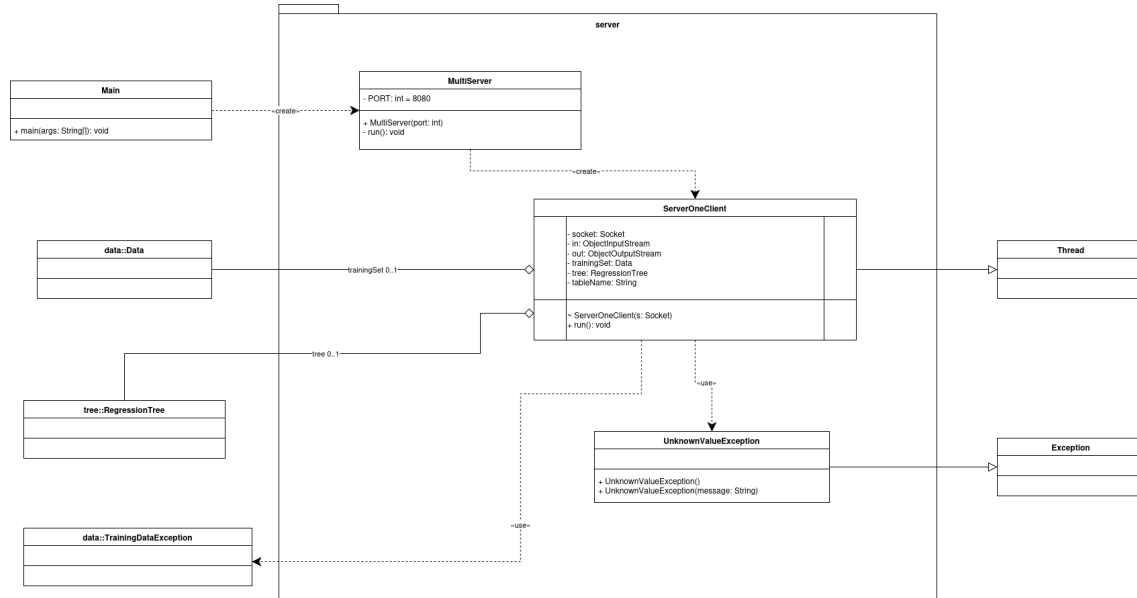


Figura 2.3: Diagramma delle classi del package **server**.

2.4 Package data

Il package **data** rappresenta gli esempi e gli attributi del training set. L'ultima colonna della tabella è interpretata come attributo target e deve essere numerica.

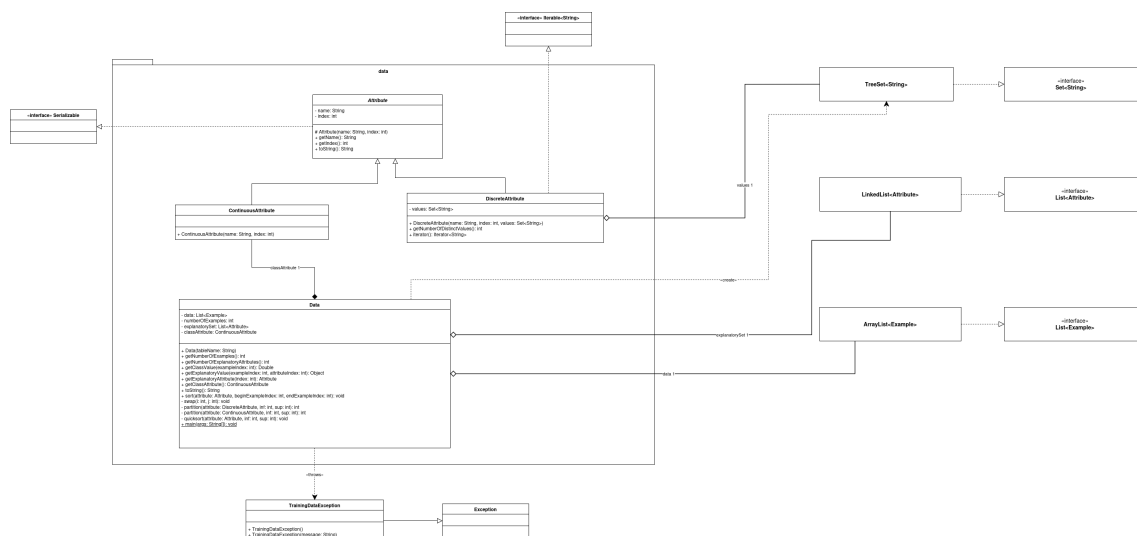


Figura 2.4: Diagramma delle classi del package **data**.

2.5 Package database

Il package `database` realizza l'accesso al DBMS tramite JDBC, legge lo schema della tabella, acquisisce le transazioni e individua i valori distinti degli attributi.

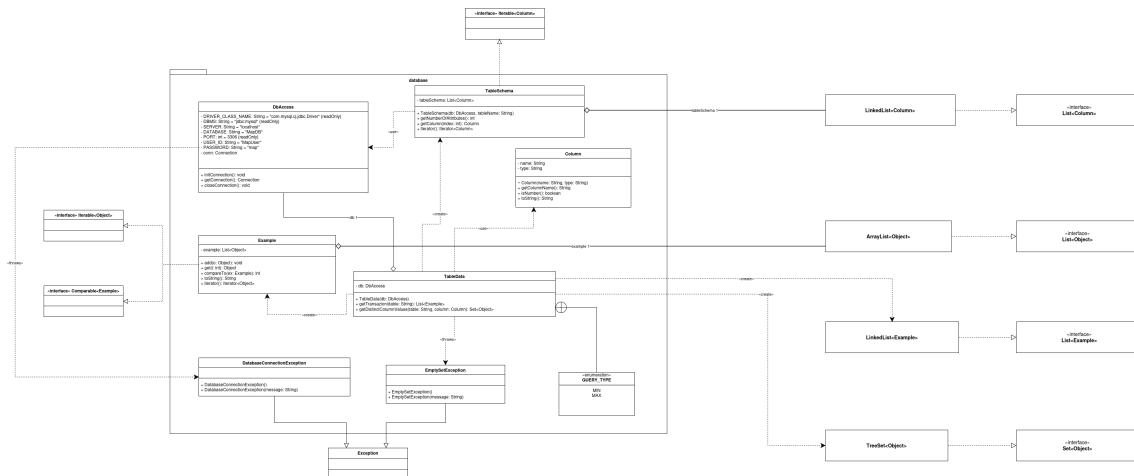


Figura 2.5: Diagramma delle classi del package `database`.

2.6 Package tree

Il package **tree** modella i nodi dell'albero di regressione e contiene le operazioni di apprendimento, serializzazione, caricamento e predizione.

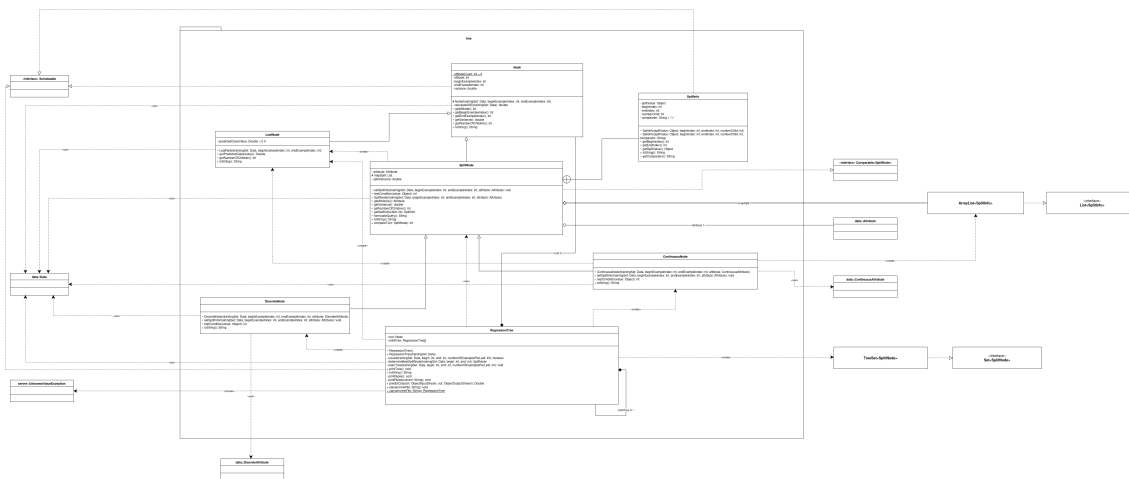


Figura 2.6: Diagramma delle classi del package `tree`.

Installazione e avvio

3.1 Preparazione

Estrarre il file `.zip` consegnato e aprire la cartella `progetto_esteso` in un terminale.

Nella cartella `dist` sono presenti i JAR necessari all'avvio del server e del bot.

L'ordine corretto di preparazione e avvio è:

1. configurare e avviare MySQL;
2. avviare il server;
3. avviare il bot;
4. aprire la chat Telegram e avviare una sessione.

3.2 Avvio di MySQL

Prima di avviare il server è necessario verificare che il servizio MySQL sia attivo e che sia disponibile un database compatibile con il progetto.

Se nel sistema sono già presenti il database, l'utente e le tabelle richieste dal progetto, è sufficiente controllare che MySQL sia in esecuzione e procedere con la sezione successiva.

La configurazione utilizzata dal progetto è la seguente:

Parametro	Valore
Database	MapDB
Utente	MapUser
Password	map
Host	localhost
Porta MySQL	3306

Il database deve inoltre contenere almeno una tabella idonea all'apprendimento di un albero di regressione. L'ultima colonna della tabella deve rappresentare l'attributo target e deve essere di tipo numerico.

3.2.1 Controllo del servizio su Linux

Su Linux Mint e sui sistemi che utilizzano `systemd`, lo stato del servizio può essere controllato con:

```
sudo systemctl status mysql
```

Se MySQL non è attivo, avviarlo con:

```
sudo systemctl start mysql
```

3.2.2 Controllo del servizio su Windows

Su Windows il nome del servizio può variare in base alla versione e al metodo utilizzato per installare MySQL. Un nome comune è `MySQL80`.

Aprire PowerShell come amministratore e cercare i servizi MySQL disponibili:

```
Get-Service -Name "MySQL*"
```

Se, ad esempio, il servizio si chiama `MySQL80`, può essere avviato con:

```
Start-Service -Name MySQL80
```

In alternativa, dal Prompt dei comandi avviato come amministratore:

```
sc query MySQL80  
net start MySQL80
```

Se il servizio presenta un nome differente, sostituire `MySQL80` con quello mostrato dal sistema.

3.2.3 Creazione automatica del database

Se la configurazione richiesta non è già presente, è possibile utilizzare lo script:

```
src/setup.sql
```

Il percorso è relativo alla cartella principale del progetto.

Lo script:

- crea il database `MapDB`, se non esiste;
- ricrea l'utente `MapUser` con password `map`;
- assegna all'utente i privilegi necessari su `MapDB`;
- crea e popola le tabelle `prova`, `provaC` e `servo`.

Nota importante. Lo script elimina e ricrea l'utente `MapUser` e le tabelle `prova`, `provaC` e `servo`. Eventuali dati già presenti in queste tabelle verranno sostituiti.

Esecuzione dello script su Linux

Aprire un terminale nella cartella principale del progetto ed eseguire:

```
sudo mysql < src/setup.sql
```

Esecuzione dello script su Windows

Su Windows, la procedura consiste nell'aprire il client MySQL e utilizzare il comando `SOURCE`.

Dal menu Start, avviare *MySQL Command Line Client*. In alternativa, aprire PowerShell o il Prompt dei comandi ed eseguire:

```
mysql -u root -p
```

Quando richiesto, inserire la password dell'utente amministratore `root`.

Dopo l'accesso al client MySQL, eseguire lo script mediante il comando `SOURCE`, indicando il percorso completo del file `src/setup.sql`:

```
SOURCE C:/percorso/completo/del/progetto_esteso/src/setup.sql;
```

Ad esempio:

```
SOURCE C:/Users/NomeUtente/Desktop/progetto_esteso/src/setup.sql;
```

Il comando deve terminare con il punto e virgola.

Se il comando `mysql` non viene riconosciuto dal terminale, è possibile avviare *MySQL Command Line Client* direttamente dal menu Start oppure eseguire `mysql.exe` indicando il percorso completo.

Al termine dell'esecuzione dello script, uscire dal client MySQL con:

```
EXIT;
```

Nota importante. Non è stato possibile verificare direttamente le procedure relative a MySQL su un sistema Windows.

3.2.4 Verifica della configurazione

Per verificare che il database e le tabelle siano stati creati correttamente, accedere con l'utente del progetto:

```
mysql -u MapUser -p -h localhost MapDB
```

Quando richiesto, inserire la password `map` e mostrare le tabelle:

```
SHOW TABLES;
```

Il risultato deve includere:

```
prova
provaC
servo
```

È possibile verificare il contenuto di una tabella, ad esempio **provaC**, con:

```
SELECT * FROM provaC;
```

Dopo il controllo, uscire dal client MySQL con:

```
EXIT;
```

3.3 Avvio del server

Prima di avviare il server, verificare che MySQL sia attivo e che il database sia correttamente configurato.

3.3.1 Avvio del server su Linux

Aprire un terminale nella cartella principale del progetto.

Alla prima esecuzione potrebbe essere necessario rendere eseguibile lo script:

```
chmod +x dist/avvia_server.sh
```

Avviare quindi il server con:

```
./dist/avvia_server.sh
```

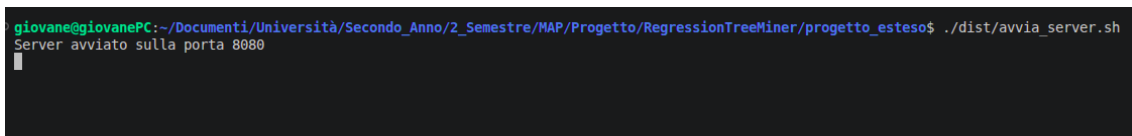
In alternativa:

```
bash dist/avvia_server.sh
```

Lo script si sposta automaticamente nella cartella **dist** ed esegue:

```
java -jar server.jar
```

Il server resta in ascolto sulla porta TCP 8080. Il terminale deve rimanere aperto durante l'utilizzo del bot.



```
giovane@giovanePC:~/Documenti/Università/Secondo_Anno/2_Semestre/MAP/Progetto/RegressionTreeMiner/progetto_esteso$ ./dist/avvia_server.sh
Server avviato sulla porta 8080
```

Figura 3.1: Avvio del server tramite script su Linux.

3.3.2 Avvio del server su Windows

Aprire il Prompt dei comandi o PowerShell nella cartella principale del progetto ed eseguire:

```
dist\avvia_server.bat
```

In alternativa, aprire la cartella `dist` tramite Esplora file e fare doppio clic su `avvia_server.bat`.

Nota importante. Non è stato possibile verificare direttamente la correttezza dello script `avvia_server.bat` su un sistema Windows.

3.4 Avvio del bot Telegram

Il bot Telegram utilizzato dal progetto è già stato creato ed è disponibile con il seguente username:

```
@RegressionTreeMinerBot
```

Il bot deve essere avviato soltanto dopo il server. Gli script forniti configurano automaticamente la connessione al server locale utilizzando l'indirizzo `127.0.0.1` e la porta `8080`.

3.4.1 Avvio del bot su Linux

Aprire un secondo terminale nella cartella principale del progetto.

Alla prima esecuzione potrebbe essere necessario rendere eseguibile lo script:

```
chmod +x dist/avvia_bot.sh
```

Avviare quindi il bot con:

```
./dist/avvia_bot.sh
```

In alternativa, eseguire lo script tramite Bash:

```
bash dist/avvia_bot.sh
```

Lo script si sposta automaticamente nella cartella `dist`. Se il file `telegram_token.txt` è presente, legge il token dalla sua prima riga; in caso contrario lo richiede tramite il terminale.

Infine, lo script esegue un comando equivalente a:

```
java -jar bot.jar "TOKEN_TELEGRAM" 127.0.0.1 8080
```


Il terminale deve rimanere aperto durante l'utilizzo del bot.

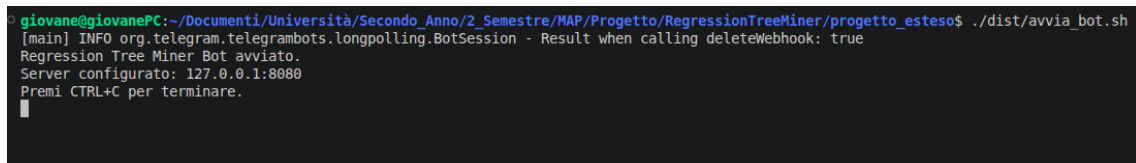


Figura 3.2: Avvio del bot Telegram tramite script su Linux.

3.4.2 Avvio del bot su Windows

Aprire il Prompt dei comandi o PowerShell nella cartella principale del progetto ed eseguire:

```
dist\avvia_bot.bat
```

In alternativa, aprire la cartella `dist` tramite Esplora file e fare doppio clic sul file:

```
avvia_bot.bat
```

Anche lo script Windows cerca il token nella prima riga del file `telegram_token.txt`. Se il file non è presente oppure è vuoto, viene richiesto di inserire il token nel terminale.

Lo script esegue un comando equivalente a:

```
java -jar bot.jar "TOKEN_TELEGRAM" 127.0.0.1 8080
```

Al termine dell'esecuzione, il comando `pause` mantiene aperta la finestra e permette di leggere eventuali messaggi o errori.

Nota importante. Non è stato possibile verificare direttamente la correttezza dello script `avvia_bot.bat` su un sistema Windows. Le immagini riportate nel manuale si riferiscono all'esecuzione su Linux.

3.4.3 Connessione del bot a un server remoto

Gli script forniti collegano automaticamente il bot al server locale `127.0.0.1:8080`.

Per collegare il bot a un server eseguito su un altro computer è necessario avviare manualmente il JAR, indicando il token, l'indirizzo IP e la porta:

```
java -jar dist/bot.jar "TOKEN_TELEGRAM" 192.168.1.50 8080
```

L'indirizzo IP mostrato è soltanto un esempio e deve essere sostituito con quello effettivo del server.

Utilizzo del bot Telegram

4.1 Avvio di una sessione

Aprire Telegram e cercare il bot mediante il seguente username:

```
@RegressionTreeMinerBot
```

Aprire la chat e premere il pulsante *Avvia*, oppure inviare:

```
/start
```

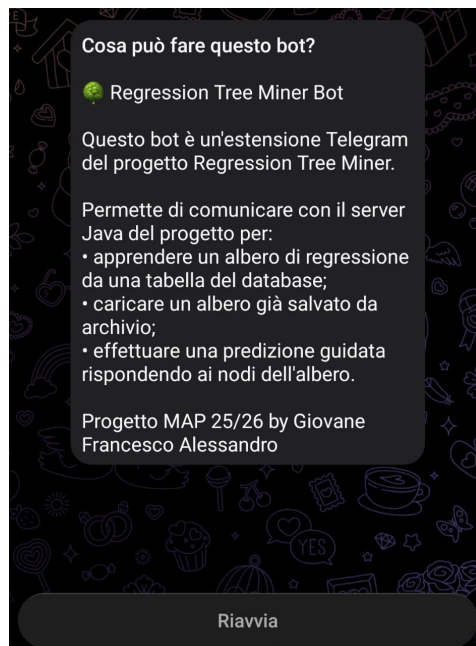


Figura 4.1: Info bot prima dell'avvio.

Il bot chiude l'eventuale sessione precedente, tenta la connessione al server e mostra il messaggio di benvenuto seguito dal menu principale:

1. apprendere un nuovo albero da una tabella del database;

2. caricare un albero già salvato da archivio.

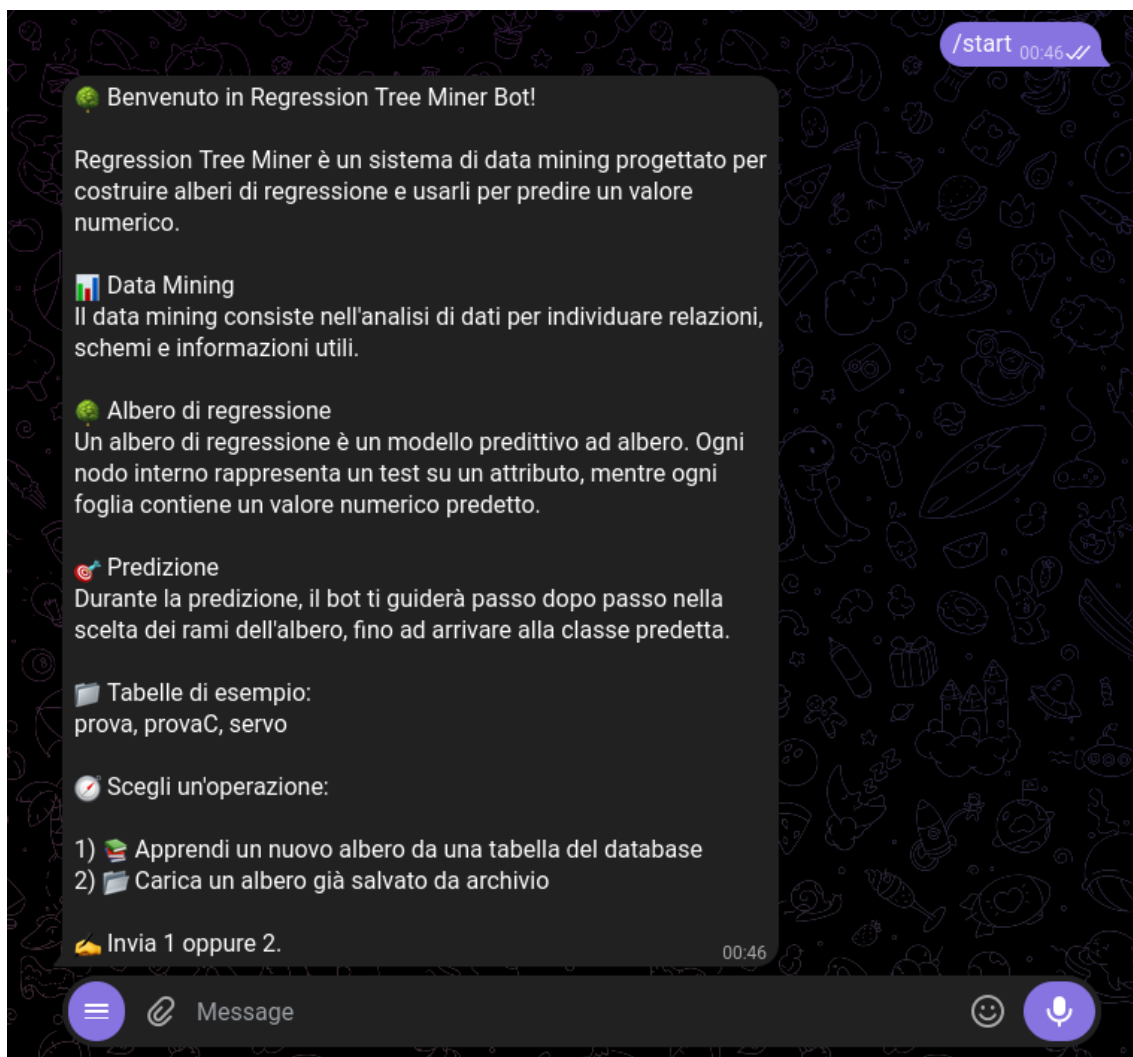


Figura 4.2: Avvio di una sessione con @RegressionTreeMinerBot e visualizzazione del menu principale.

Se il server non è raggiungibile, il bot segnala l'impossibilità di stabilire la connessione. Dopo aver avviato o ripristinato il server, inviare nuovamente `/start`.

4.2 Comandi disponibili

Comando	Funzione
<code>/start</code>	Avvia una nuova sessione e mostra il menu principale.
<code>/end</code>	Termina la sessione corrente e chiude la connessione con il server.
<code>/help</code>	Mostra la guida all'utilizzo del bot.
<code>/info</code>	Mostra informazioni sul progetto e sulla sua architettura.

I comandi `/help` e `/info` possono essere utilizzati anche senza una sessione attiva, purchè il `bot.jar` sia in esecuzione.

4.3 Apprendimento di un nuovo albero

Per apprendere un albero tramite Telegram:

1. inviare `/start`;
2. inviare `1`;
3. inserire il nome della tabella, ad esempio `provaC`;
4. attendere il caricamento del training set e l'apprendimento;
5. verificare il messaggio che conferma il salvataggio dell'albero;
6. proseguire con la predizione guidata.

L'albero viene salvato dal server nella sua directory di lavoro con un nome derivato dalla tabella, ad esempio:

```
provaC.dmp
```

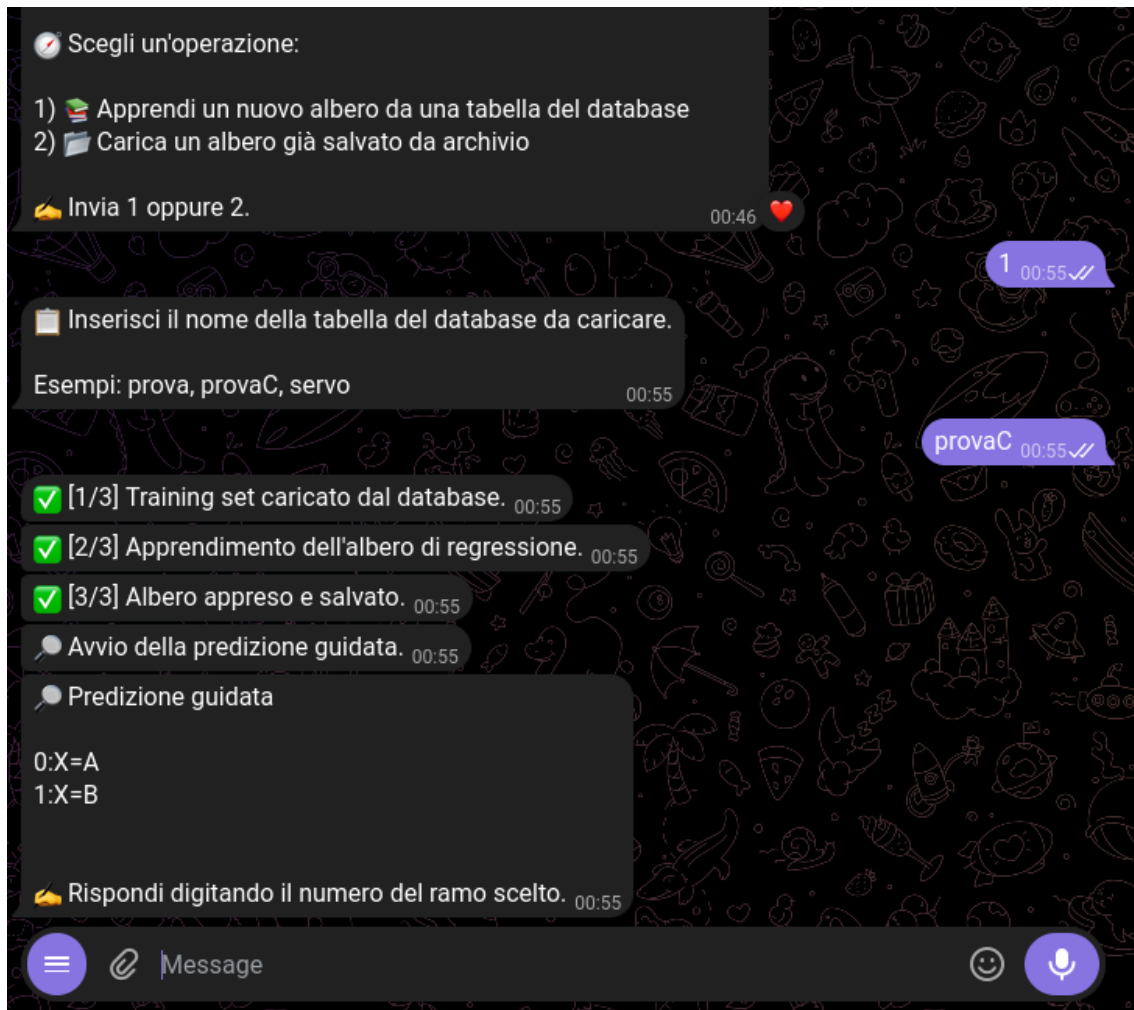


Figura 4.3: Apprendimento di un nuovo albero attraverso la chat Telegram.

4.4 Predizione guidata

Durante la predizione il bot mostra una domanda relativa a un nodo dell'albero e associa un numero a ogni ramo disponibile.

L'utente deve rispondere inviando esclusivamente il numero del ramo scelto. Se viene inserito un testo non numerico o un valore non ammesso, il bot mostra un messaggio di errore e ripropone la domanda corrente.

Il procedimento continua fino al raggiungimento di una foglia, quando il bot mostra la classe numerica predetta.

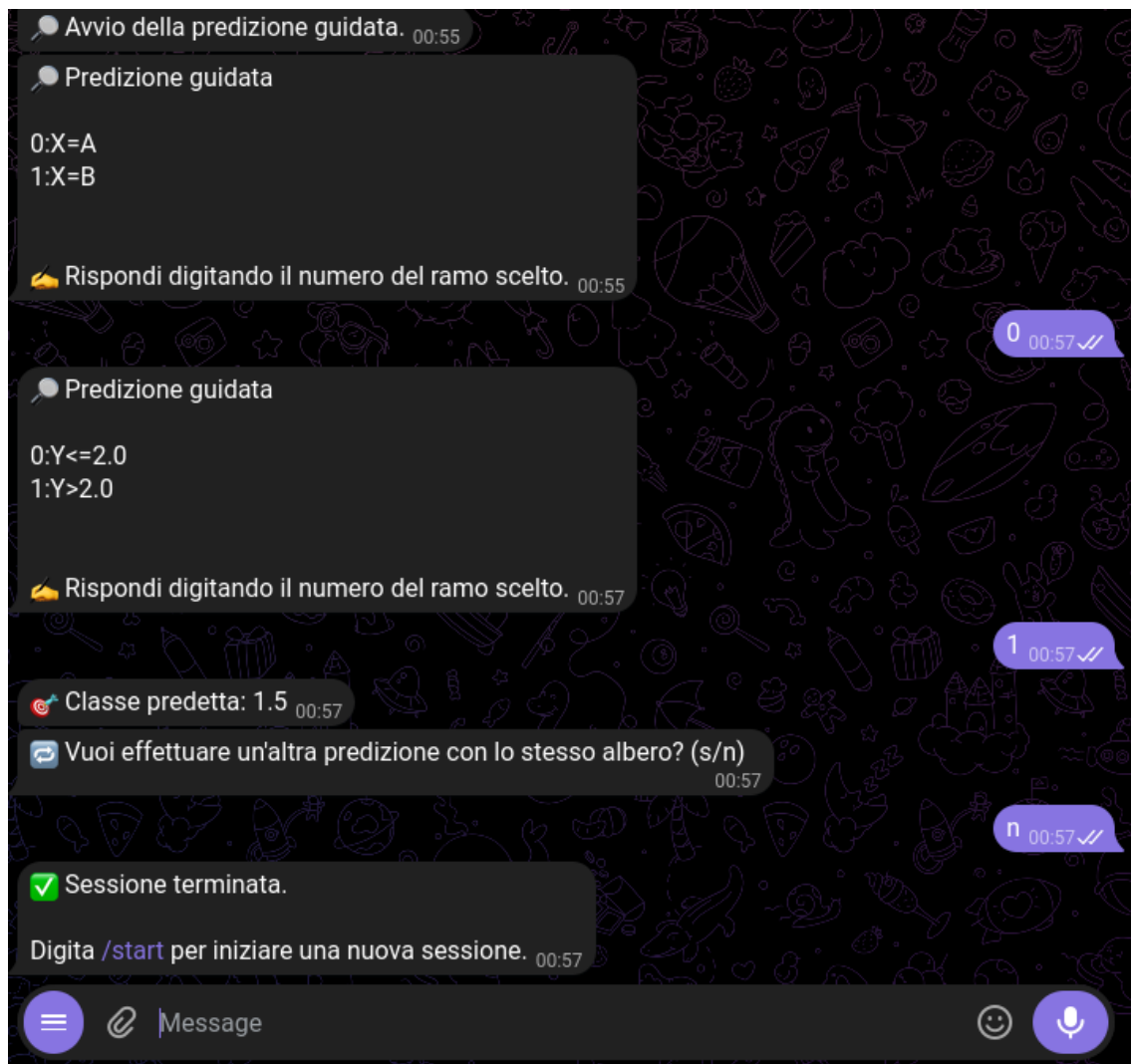


Figura 4.4: Predizione guidata e visualizzazione della classe predetta.

Dopo la predizione, il bot permette di:

- effettuare un'altra predizione utilizzando lo stesso albero;
- chiudere la sessione corrente.

4.5 Caricamento di un albero salvato

Per utilizzare un albero già serializzato:

1. verificare che il file `.dmp` si trovi nella directory di lavoro del server;
2. inviare `/start`;
3. inviare `2`;
4. inserire il nome dell'archivio senza l'estensione `.dmp`, ad esempio `provaC`;

5. attendere il messaggio di conferma;
6. completare la predizione guidata.

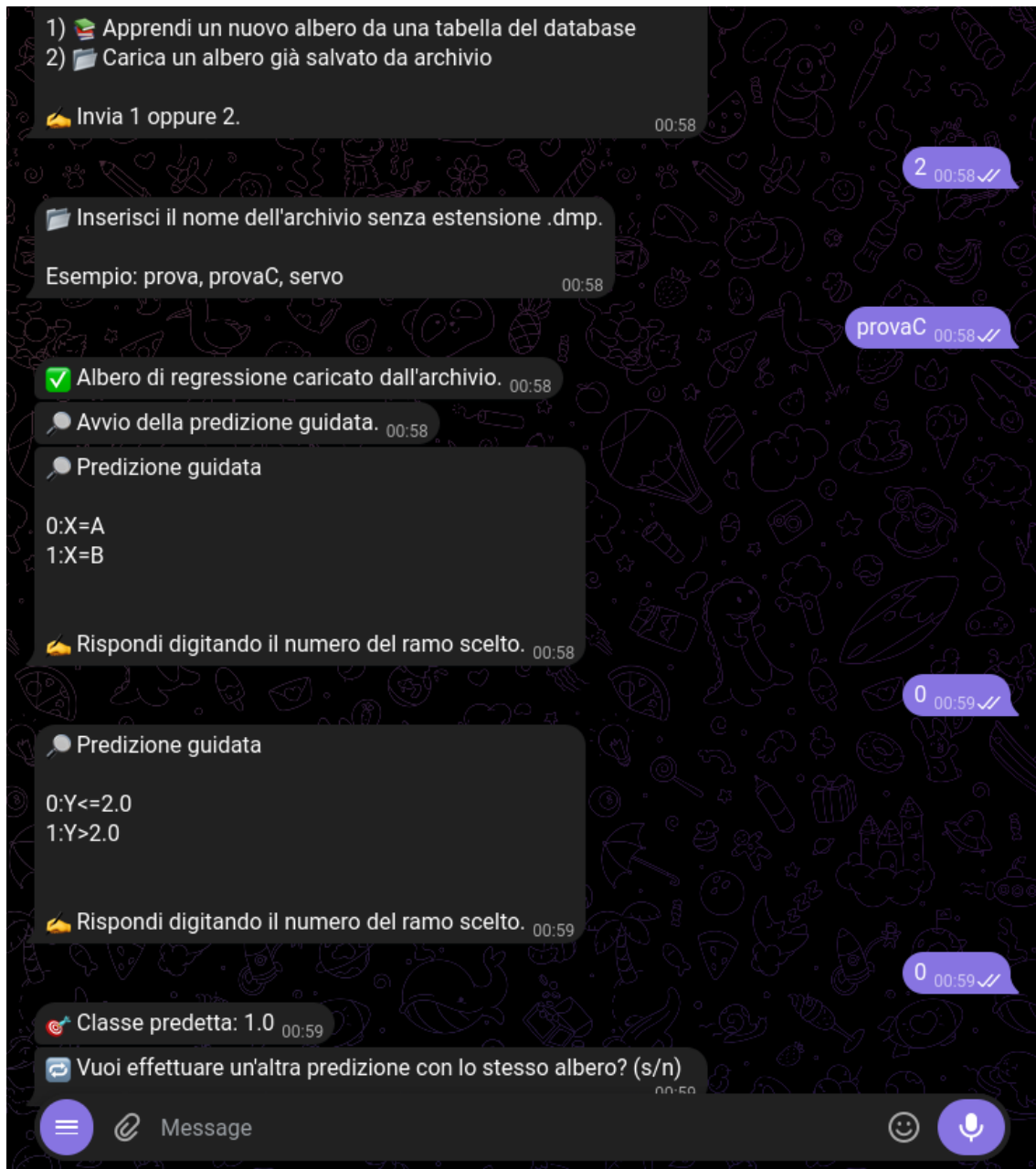


Figura 4.5: Caricamento di un albero serializzato tramite Telegram.

4.6 Chiusura della sessione Telegram

Per terminare la sessione corrente nel chat bot, inviare:

```
/end
```

La chiusura della sessione non arresta il processo del bot e non interrompe le sessioni appartenenti ad altre chat.

4.7 Utilizzo del comando `/help`

Il comando `/help` mostra una guida sintetica all'utilizzo del bot e riepiloga le operazioni disponibili.

Per visualizzare la guida, inviare nella chat:

```
/help
```

Il bot risponde mostrando i principali comandi supportati e le indicazioni necessarie per avviare o terminare una sessione.

Il comando può essere utilizzato anche quando non è attiva una sessione con il server.

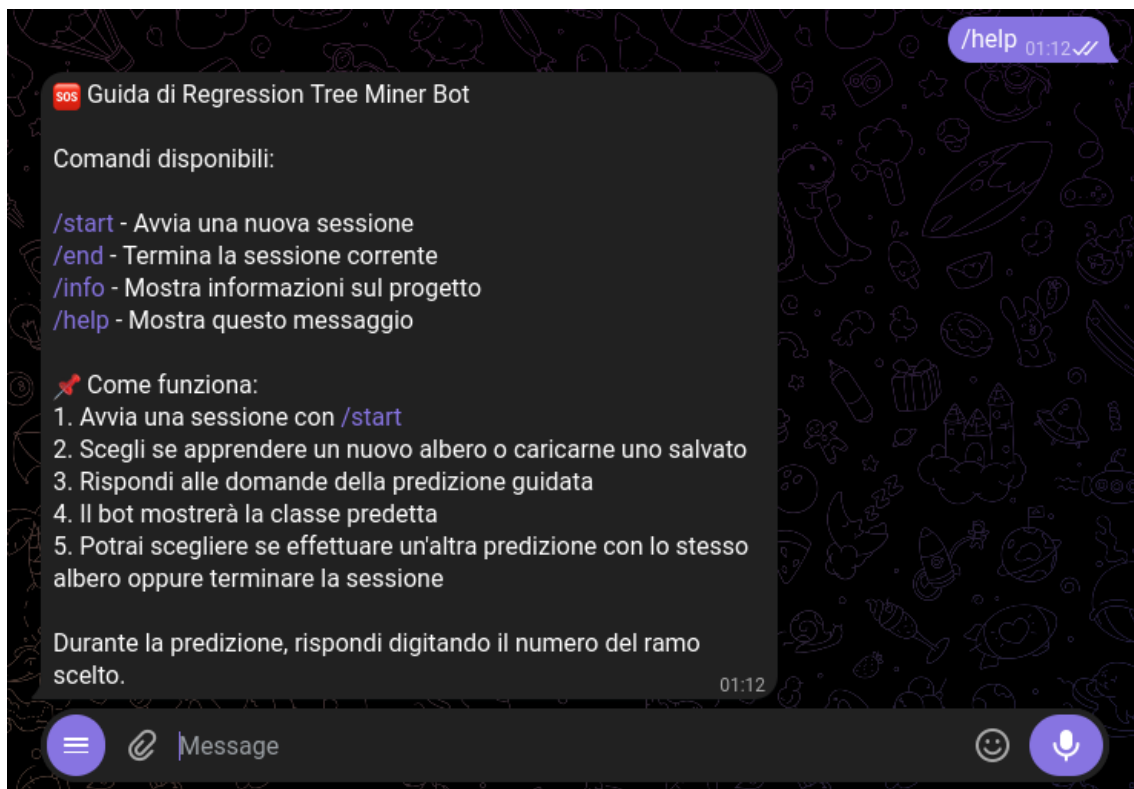


Figura 4.6: Visualizzazione della guida tramite il comando `/help`.

4.8 Utilizzo del comando `/info`

Il comando `/info` mostra informazioni generali sul progetto *Regression Tree Miner* e sull'architettura utilizzata.

Per visualizzare le informazioni, inviare nella chat:

```
/info
```

Il bot risponde con una breve descrizione del progetto e della comunicazione tra il bot Telegram e il server.

Anche questo comando può essere utilizzato senza avviare preventivamente una sessione con il comando `/start`.

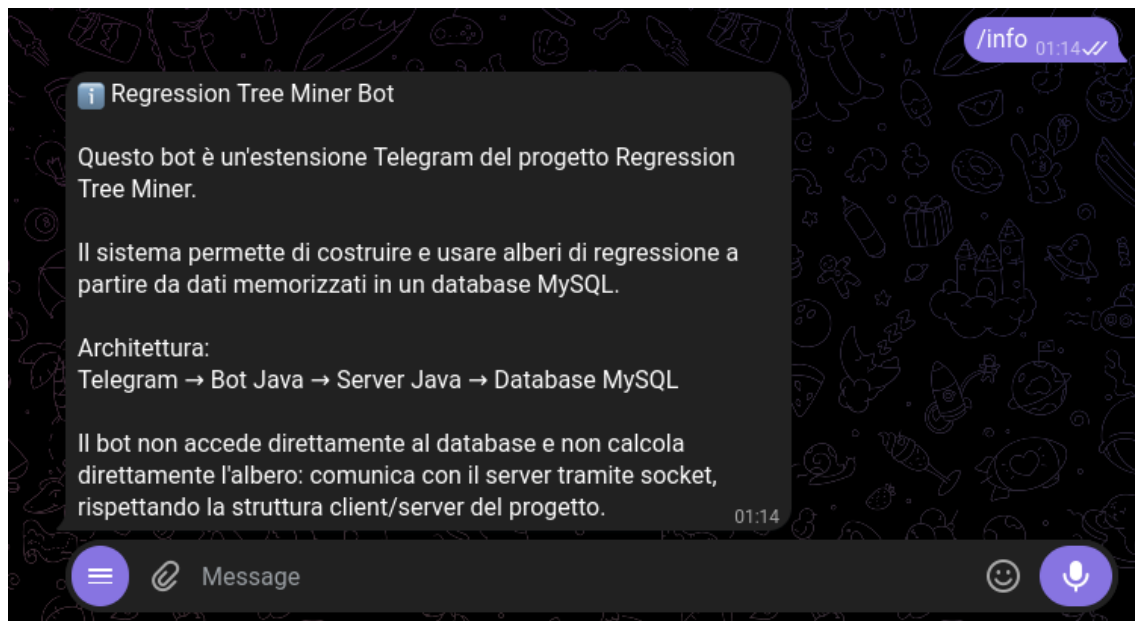


Figura 4.7: Visualizzazione delle informazioni sul progetto tramite il comando `/info`.

Chiusura dell'applicazione

5.1 Arresto del bot

Nel terminale nel quale è in esecuzione `bot.jar`, premere:

```
CTRL+C
```

L'arresto del bot impedisce agli utenti Telegram di continuare a utilizzare il servizio fino al successivo avvio.

5.2 Arresto del server

Dopo aver arrestato il bot, nel terminale del server premere:

```
CTRL+C
```

È consigliabile arrestare il server solo dopo la chiusura delle sessioni attive.

5.3 Conservazione degli alberi

I file `.dmp` prodotti dal server devono essere conservati se si desidera riutilizzare gli alberi senza ripetere l'apprendimento. Prima di spostare il progetto, verificare la directory dalla quale è stato avviato `server.jar`, perché gli archivi vengono creati rispetto alla directory di lavoro del server.